

A Low Complexity 32-Point FFT Architecture Using Input Grouping, Partial Sum Sharing, and Clock Gating Technique

Dr. V. V. K. D. V. Prasad¹, Vaddimukkala Venu², Saniya Begum³, Yasam Gopalakrishna⁴, Rokkam Varshitha⁵

¹Professor & Convener (IQAC), Department of Electronics and Communication Engineering,
Seshadri Rao Gudlavalleru Engineering College, Gudlavalleru, Andhra Pradesh, India.

Email: varrevkdvp@rediffmail.com

²³⁴⁵UG Students, Department of Electronics and Communication Engineering,
Seshadri Rao Gudlavalleru Engineering College, Gudlavalleru, Andhra Pradesh, India.

² venuvaddimukkala9@gmail.com, ³ saniyabegum79814@gmail.com, ⁴ yasamgopalakrishna@gmail.com, ⁵ rokkamvarshitha@gmail.com

Abstract: The Fast Fourier Transform (FFT) is a fundamental algorithm widely used in digital signal processing for efficient frequency-domain analysis. However, conventional FFT architectures suffer from high computational complexity, increased power consumption, and large hardware requirements. This paper proposes a low-complexity 32-point FFT architecture based on an innovative input grouping and partial sum sharing technique. The proposed method reduces redundant arithmetic operations by grouping inputs with similar twiddle factors and reusing intermediate results. Additionally, a flip-flop-based clock gating technique is incorporated to minimize dynamic power consumption by disabling inactive clock regions. The architecture is implemented using Verilog HDL and validated using Xilinx Vivado. Experimental results demonstrate a significant reduction in power consumption and delay with only a marginal increase in area. The proposed design is suitable for real-time and low-power applications such as wireless communication, IoT, and embedded systems.

Keywords: FFT, Input Grouping, Partial Sum Sharing, Clock Gating, Low Power, FPGA

1. Introduction

In modern digital signal processing systems, the Fast Fourier Transform (FFT) plays a vital role in transforming time-domain signals into frequency-domain representations. This transformation is essential in applications such as orthogonal frequency division multiplexing (OFDM), image enhancement, spectrum analysis, and biomedical signal processing. The FFT reduces the computational burden of the Discrete Fourier Transform (DFT) from quadratic complexity $O(N^2)$ to logarithmic complexity $O(N \log N)$, making it suitable for real-time processing. However, despite this reduction, practical hardware implementations of FFT still involve a large number of multipliers, adders, and memory elements. These components contribute significantly to power consumption and silicon area, especially in embedded and portable devices.

With the advent of portable and battery-operated devices, the need for a low power and high performance FFT processor is on the rise. Conventional FFT structures perform redundant operations and work under full clock activity even if some of the blocks are idle. This results in unnecessary power dissipation. There is a need to reduce the arithmetic complexity, minimize redundant operations, and reduce the dynamic power. The conventional FFT architectures are facing many challenges like a large number of complex multiplications, redundant operations, continuous switching of the clocks leading to the loss of dynamic power, and delay due to poor data flow.

The objective of this work is to design an efficient FFT architecture that reduces computational complexity using input grouping, minimizes hardware usage using partial sum sharing, reduces

power consumption using clock gating technique and achieves better delay performance. The key contributions of this paper are:

1. A novel input grouping mechanism to reduce multipliers
2. Efficient partial sum reuse strategy
3. Integration of flip-flop-based clock gating
4. FPGA-based implementation and performance evaluation
5. Comparative analysis demonstrating improved efficiency

2. Literature Review

The N-point DFT $X(k)$ of $x(n)$ is described using equation (1).

$$X(k) = \sum_{n=0}^{N-1} x(n)W_N^{nk}, k = 0 \text{ to } N - 1 \quad (1)$$

Where $x(n)$ is the discrete time is input sequence and $X(k)$ is its frequency domain representation. $W_N^{nk} = e^{-j2\pi nk/N}$ is known as the twiddle factor.

Computing DFT directly using equation (1) involves N^2 multiplications and $N(N - 1)$ additions, which are computationally intensive operations, especially for large N. Hence, FFT was proposed to efficiently compute DFT. Cooley-Tukey radix-2 decimation in time (DIT) FFT is the simplest and most popular algorithm and is defined as:

$$X(k) = X_1(k) + W_N^k X_2(k) \quad (2)$$

$$X_1(k) = \sum_{n=0}^{\frac{N}{2}-1} x(2n)W_N^{nk} \quad (3)$$

$$X_2(k) = \sum_{n=0}^{\frac{N}{2}-1} x(2n + 1)W_N^{nk} \quad (4)$$

By recursively using equation (2), the DFT of an N-point signal can be broken down into $\log_2 N$ stages, and in each stage, there are N/2 two-point DFTs. The computation scheme of a 16-point decimation-in-time Fast Fourier Transform (DIT-FFT) in four stages is demonstrated in Figure 1.

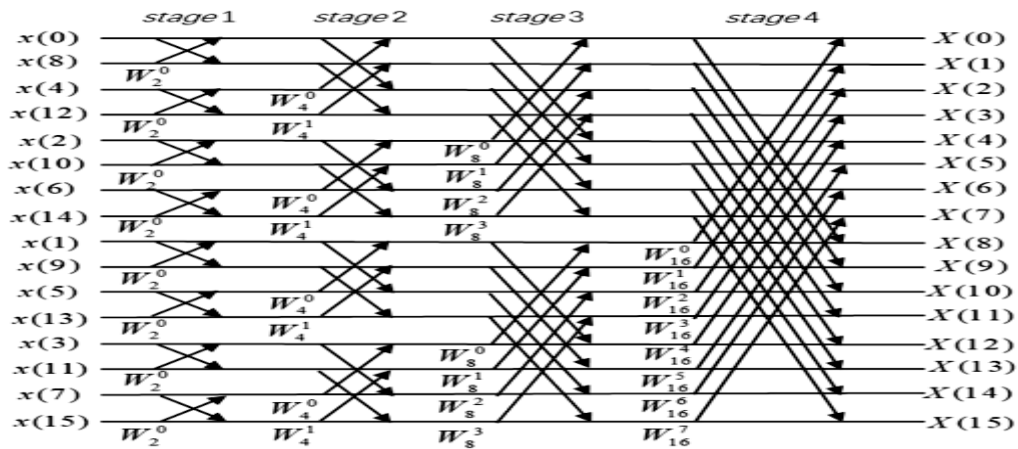


Figure 1: Implementation of 16-point Decimation in Time FFT algorithm

FFT was mostly implemented in software. However, with the introduction of system-on-a-chip (SoC) and other techniques, the implementation of FFT in hardware becomes possible in order to reduce the area and power consumption under the circumstances of maintaining the output throughout the process. The existing architectures in the implementation of FFT are mostly categorized into two types: memory-based architectures [6]-[8] and pipelined architectures [9]-[11]. The first type of architecture utilizes a single butterfly processing element (PE) and memory blocks in the implementation of the FFT algorithm. The utilization of the computing resources is reduced in this architecture, but the data throughput is low in this architecture. For applications that require higher throughputs, such as IEEE 802.11ad [12], the pipelined architectures are mostly used in the implementation of the FFT algorithm. These pipelined architectures can be further classified into single-path delay feedback (SDF) [13], [14], multi-path delay feedback (MDF) [15] and multipath delay commutator (MDC) [16].

Because of the need to add more butterfly PEs and FFT rotators to execute twiddle factors' multiplication, the hardware cost of the pipelined architectures is substantially increased. To overcome this problem, CSD arithmetic and common sub-expression sharing techniques have been proposed. In order to reduce the number of adders needed in the hardware implementation of the FFT architecture, the authors of [17] have proposed an efficient parallel FFT architecture that can be achieved by truncating the insignificant bits. A new approximate parallel FFT architecture has been proposed recently in [18] where a new twiddle factors merging technique is presented to execute the merged expressions. Although the low area-power design is achieved in [18], more efficient hardware can be achieved within the margin of error.

3. Proposed Method

In this paper, a new 32-point Fast Fourier Transform (FFT) architecture is introduced, which is based on a 16-point FFT as the fundamental computing component and building block. Instead of designing a 32-point FFT, which involves a larger number of arithmetic operations, hardware resources, and power consumption, the design is broken down into two 16-point FFT operations, making the design much easier and reducing the number of complex multiplications and additions. This not only reduces the complexity of the design but also aids the efficient use of hardware resources like adders, multipliers, and memory. Furthermore, since the 16-point FFT module has already been optimized using techniques like input grouping and partial sum sharing, the 32-point FFT module inherits the advantages of the 16-point FFT module, thus improving the performance. However, it is observed that the power consumption of the circuit increases due to the continuous switching of the clocks. This problem can be solved by implementing a clock gating technique.

3.1 Flip flop-based clock gating technique

In various applications, latch-based designs are converted to flip-flop-based designs. The flip-flop is divided, creating two latches from the master-slave flip-flop. This results in a D flip-flop with an AND gate. The flip-flop-based clock gating technique is used to minimize dynamic power consumption in digital systems by gating the clock during idle periods in the system. This technique differs from latch-based clock gating, as it provides a glitch-free gated clock through synchronization of the enable signal (En) with the clock edge, as achieved through flip-flop-based clock gating. This technique samples the enable signal using a D flip-flop on the rising edge of the clock signal. The output of the flip-flop is then performed logical AND operation with the clock signal to generate a gated clock (CLK_GATED) as shown in figure 2.

This ensures that the gated clock transitions only occur at well-defined clock edges, preventing glitches and ensuring reliable operation.

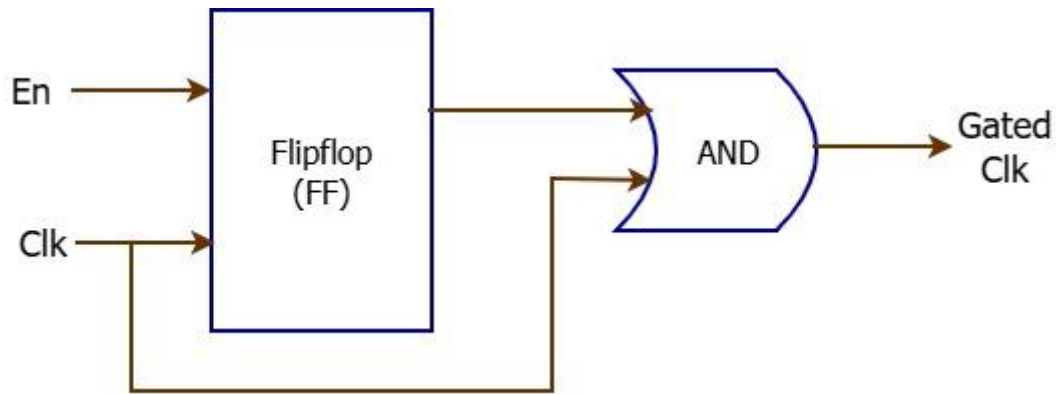


Figure 2: Flip flop-based clock gating technique

As depicted in Table 1, the gated clock will be driven to a high level if the output of the flip-flop and the clock signal are both high. Otherwise, the gated clock will stay in the zero state. It can be concluded that, if the clock signal is driven to the sleep mode, the gated clock will stay in the zero state. This method can be used to prevent the enable signal from being sampled on the falling edge of the clock signal. This method is commonly applied to low-power processors, digital signal processing circuits, and FPGA/ASIC designs. Compared to the latch-based method of clock gating, the flip-flop method provides better timing closure, solves the problem of race conditions, and can be easily incorporated into a synchronous design.

Table 1: Truth Table of Clock Gating

Clock (Clk)	Enable(En)	FF output	Gated Clk	Operation
0	0	0	0	Clk OFF
1	0	0	0	Clk OFF
0	1	1	0	Waiting
1	1	1	1	Clk ON

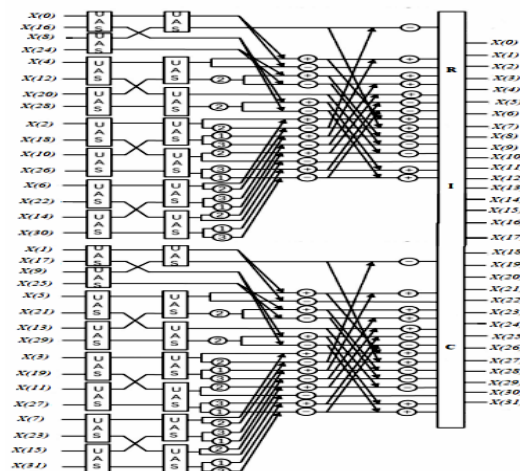


Figure 3: Proposed 32-point FFT architecture

The proposed 32-point FFT architecture using clock gating logic of figure 2 is shown in figure 3. The architecture is divided into three main parts:

1. *Left Side (Input Section)*: Inputs are labeled as: $X(0), X(1), X(2), \dots, X(31)$. These inputs are arranged in groups (not processed individually like traditional FFT)

2. *Middle Section (Computation Core)* : The unit ADD or SUBTRACT block (UAS) acts like simplified butterfly units, which are used in the design to combine input pairs and reduce number of operations before multiplication. After UAS operations, intermediate results are reused multiple times to avoid re-computing the same values and extra adders and multipliers. The criss-cross lines in the middle indicate data routing between stages and proper alignment of FFT outputs, thus replacing complex memory-based data shuffling and traditional stage-wise FFT connections.

3. *Right Side (Output Section)*: Final FFT outputs are produced from this stage, which are separated into real and imaginary parts.

4. Results and Discussion

4.1 Implementation of Proposed Architecture using Verilog HDL

The 32-point Fast Fourier Transform (FFT) architecture and the flip-flop-based clock gating technique are implemented using the Verilog HDL programming language. The architecture is based on a modular approach, in which each component is designed as a separate module and then combined to form the entire system. The top-level module, named `top.v`, is the main controller and connects all the modules together and controls the flow of data in the entire system. The module controls the input signals, clock, and enables signals required for the entire system. The 32-point FFT module is designed as a separate module named `top_32.v` and is based on a divide-and-conquer approach using two 16-point FFT blocks. The input data is split into even and odd values and is then processed independently in the 16-point FFT block.

The "Unit Arithmetic Structure (UAS.v)" module is used for arithmetic operations such as addition and subtraction that are needed for butterfly computation. The module is used for both real and imaginary parts of the Fast Fourier Transform (FFT). Therefore, it is possible to handle complex numbers. The "multiplexor (mux_2to1.v)" module is used to control the selection of data paths of input information at different stages of computation. To make the design more power-efficient, a "flip-flop-based clock gating (flipflop_based_clk.v)" module is used. The clock is controlled through this module's enable signal. The clock is enabled only during computation and disabled during idle time. This reduces unnecessary switching of the clock and hence minimizes dynamic power dissipation. The design is simulated and verified. The "RTL Schematics" of the design is created to ensure that the hardware is correctly structured. The "Simulation waveforms" of the design are also analyzed to ensure that it is functioning correctly. The addition of the clock gating feature to the FFT architecture makes it more power-efficient.

The schematic of the Register Transfer Level (RTL) of the suggested 32-point Fast Fourier Transform (FFT) architecture, as designed using the Xilinx Vivado tool after the synthesis of the Verilog code, is presented in Figure 4. The schematic of the designed architecture is a high-level diagram that indicates the interconnections between the modules of the architecture,

including the 16-point FFT blocks, the Unit Arithmetic Structure (UAS), the multiplexers, and the clock gating block. The schematic indicates that the 32-point FFT architecture is designed as a modular architecture that includes two 16-point FFT modules, where the input signals are distributed among the modules and the outputs are integrally connected through the interconnections of the architecture. The flow of the data among the modules is clearly indicated in the schematic, which shows the flow of the signals among the successive computation blocks of the architecture. The clock gating block is included in the architecture, and the clock signal is controlled through the enable signal so that the clock is active only during the computation process. The schematic of the designed architecture is an assurance of the logical correctness of the designed architecture and its adherence to the suggested architecture before moving further to the implementation process.



Figure 4: RTL Schematic of Proposed 32-point FFT Architecture

4.1.1 Simulation Waveform

The simulation waveform in Figure 5 shows the functional verification of the 32-point FFT architecture without clock gating. The input signals include the real and imaginary parts of the signals (A0_real, A0_imag, ...), and the output signals include the real and imaginary parts of the signals (X15_real, X15_imag, ...). The waveform shows the stability of the signals during the simulation. The absence of clock gating causes the clock signal to be always present, thus ensuring the registers switch continuously during the simulation time. The outputs show the correctness of the design.

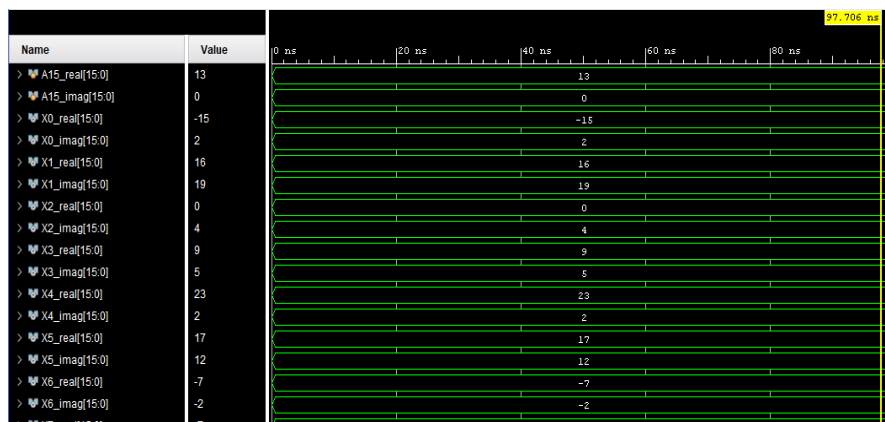


Figure 5: Simulation of 32-Point FFT Architecture Without Clock Gating

4.1.2 Area Utilization

The area report defines the hardware resources used in the design is shown in figure 6. The 32-point FFT architecture utilizes approximately 3290 Slice LUTs and 2048 bonded IOBs. The lack of clock gating implementation leads to no optimization in the registers and logic reduction, causing higher utilization in this case.

Name ¹	Slice LUTs (134600)	Bonded IOB (400)	
N top_32	3290	2048	

Figure 6: Device Utilization

4.1.3 Delay Analysis

The delay analysis report in table 2 shows the timing performance of the 32-point FFT design. The critical path delay observed is approximately 10.561 ns, which includes logic delay (~5.610 ns) and net delay (~4.951 ns). The slack value is positive, indicating that the design meets timing requirements. However, due to the absence of clock gating, unnecessary switching contributes indirectly to timing inefficiencies in larger designs.

Table 2: Timing Analysis Report

Name	Slack ¹	Levels	Routes	High Fanout	From	To	Total Delay	Logic Delay	Net Delay
↳ Path 1	∞	13	11	33	A11_imag[3]	X15_imag[15]	10.561	5.610	4.951
↳ Path 2	∞	13	11	33	A27_imag[3]	X23_imag[15]	10.561	5.610	4.951
↳ Path 3	∞	13	11	33	A27_imag[3]	X31_imag[15]	10.561	5.610	4.951
↳ Path 4	∞	13	11	33	A11_imag[3]	X7_imag[15]	10.561	5.610	4.951
↳ Path 5	∞	13	11	33	A11_imag[3]	X15_imag[13]	10.556	5.605	4.951
↳ Path 6	∞	13	11	33	A27_imag[3]	X23_imag[13]	10.556	5.605	4.951
↳ Path 7	∞	13	11	33	A27_imag[3]	X31_imag[13]	10.556	5.605	4.951
↳ Path 8	∞	13	11	33	A11_imag[3]	X7_imag[13]	10.556	5.605	4.951
↳ Path 9	∞	13	11	33	A11_imag[3]	X15_imag[14]	10.496	5.545	4.951
↳ Path 10	∞	13	11	33	A27_imag[3]	X23_imag[14]	10.496	5.545	4.951

4.1.4 Power Consumption

From the power analysis report shown in figure 7, we can observe that the total on-chip power consumption is approximately 1000.115 W, which is significantly increased. The junction temperature is also maximized to 125°C. There is no clock gating, and thus all modules are running all the time. This is a major drawback of the current design and confirms that clock gating is a necessary power optimization technique.

Power estimation from Synthesized netlist. Activity derived from constraints files, simulation files or vectorless analysis. Note: these early estimates can change after implementation.

Total On-Chip Power:	1000.115 W (Junction temp exceeded!)
Design Power Budget:	Not Specified
Power Budget Margin:	N/A
Junction Temperature:	125.0°C
Thermal Margin:	-1809.8°C (-966.8 W)
Effective SJA:	1.9°C/W
Power supplied to off-chip devices:	0 W
Confidence level:	Low
Launch Power Constraint Advisor to find and fix invalid switching activity	

Figure 7: Power Analysis Report

4.2 Analysis of 32- Point FFT with Clock Gating

4.2.1 Simulation Waveform:

The simulation waveform of the proposed 32-point FFT architecture demonstrates the functionality of the design with clock gating enabled. The presence of the enable (en) signal controls the clock activity, allowing the circuit to operate only when required. Initially, undefined states (X) are observed, which later settle to valid values after proper signal propagation. Compared to the existing design, switching activity is reduced because the clock is gated when the enable signal is inactive. This confirms correct functional behavior along with improved power efficiency.

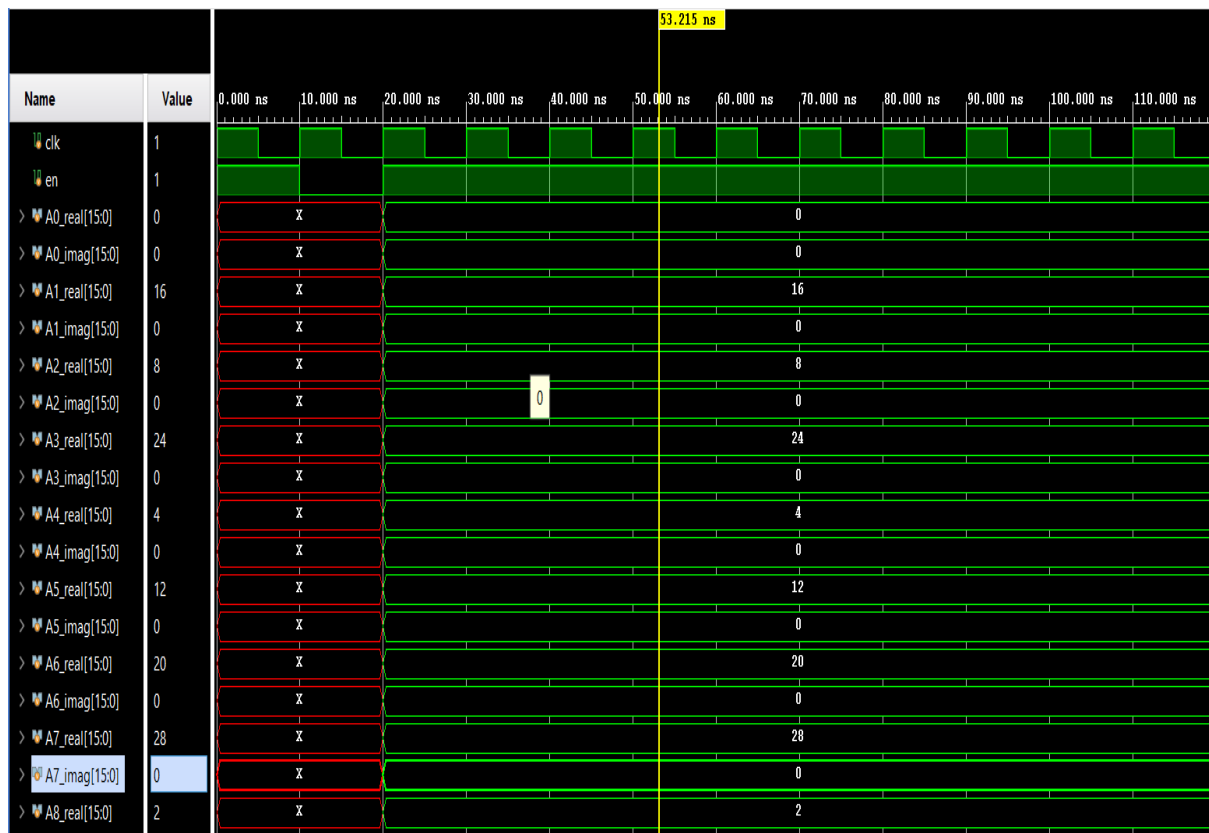


Figure 8: Simulation Waveform of 32-Point FFT with Clock Gating

4.2.2 Area Utilization

The area utilization report indicates that the proposed design uses approximately 3307 Slice LUTs and 897 Slice Registers. Compared to the existing design, there is a slight increase in LUT usage due to additional clock gating logic. However, this increase is minimal and acceptable considering the significant power savings achieved.

Name	^1	Slice LUTs (134600)	Slice Registers (269200)
▼ N top_32		3307	897
> I f1 (flipflop_based_clk)		0	1
I u1 (top_16)		1504	448
I u2 (top_16_0)		1504	448

Figure 9: Device Utilization

4.2.3 Delay Analysis

The delay analysis for the proposed design in table 3 shows that there is an improved critical path delay, which is around 7.406 ns, significantly less than that for the existing design, which is around ~10.561 ns. The logic delay is around 3.100 ns, while the net delay is around 4.306 ns. The delay has been reduced due to optimal switching activities and better management of signal propagation with clock gating techniques. The timing constraints are met with a positive slack.

Table 3: Timing Analysis Report

Name	Slack ^1	Levels	Routes	High Fanout	From	To	Total Delay	Logic Delay	Net Delay
↳ Path 1	∞	12	9	33	A11_imag[3]	u1/X7_imag_reg[13]/D	7.406	3.100	4.306
↳ Path 2	∞	12	9	33	A27_imag[3]	u2/X7_imag_reg[13]/D	7.406	3.100	4.306
↳ Path 3	∞	12	9	33	A11_imag[3]	u1/X7_imag_reg[15]/D	7.404	3.098	4.306
↳ Path 4	∞	12	9	33	A27_imag[3]	u2/X7_imag_reg[15]/D	7.404	3.098	4.306
↳ Path 5	∞	12	9	33	A11_imag[3]	u1/X7_imag_reg[14]/D	7.343	3.037	4.306
↳ Path 6	∞	12	9	33	A27_imag[3]	u2/X7_imag_reg[14]/D	7.343	3.037	4.306
↳ Path 7	∞	12	9	33	A11_imag[3]	u1/X7_imag_reg[12]/D	7.320	3.014	4.306
↳ Path 8	∞	12	9	33	A27_imag[3]	u2/X7_imag_reg[12]/D	7.320	3.014	4.306
↳ Path 9	∞	11	9	33	A11_imag[3]	u1/X7_imag_reg[9]/D	7.308	3.002	4.306
↳ Path 10	∞	11	9	33	A27_imag[3]	u2/X7_imag_reg[9]/D	7.308	3.002	4.306

6.2.4 Power Consumption

From the power analysis report shown in figure 10, it is observed that the total on-chip power consumption is minimized to approximately 62.447 W. This is a significant improvement when compared to the existing design, which consumes approximately 1000 W. Although the junction temperature is maintained at 125°C, the total power consumption is significantly minimized due to a reduction in switching activity. Clock gating is effective in disabling

modules that are not in use.

Total On-Chip Power:	62.447 W (Junction temp exceeded!)
Design Power Budget:	Not Specified
Power Budget Margin:	N/A
Junction Temperature:	125.0°C
Thermal Margin:	-56.7°C (-29.1 W)
Effective θ_{JA} :	1.9°C/W
Power supplied to off-chip devices:	0 W
Confidence level:	Low

Figure 10: Power Analysis Report

4.3 Comparative Analysis of 32-Point FFT with and without Clock Gating

The comparison between the existing 32-point FFT architecture without clock gating and the proposed architecture incorporating clock gating is carried out based on the essential parameters like area, power consumption, and delay, and the results are presented in Table 4. From the results, it is clear that the area utilization increases slightly from 3290 LUTs to 3307 LUTs in the proposed architecture, which is due to the introduction of clock gating logic in the design. A significant improvement is noted in the power consumption aspect, as the power consumption is reduced significantly from 1000.115 W in the existing architecture to 62.447 W in the proposed architecture.

As far as the delay is concerned, the proposed architecture shows significant improvement in the delay aspect, as the delay is reduced significantly from 10.561 ns in the existing architecture to 7.406 ns in the proposed architecture. From the results, it is clear that the 32-point FFT architecture incorporating clock gating is a power-efficient and faster architecture compared to the existing architecture, with a slight increase in the area utilization aspect. From the results, it is clear that clock gating is an effective technique in the optimization of digital architectures, and the technique is useful in the power-constrained applications of digital systems.

Table 4: Comparative Analysis of 32-Point FFT with and without Clock Gating

S.No	Method	Area (LUT)	Power(W)	Delay(ns)
1	Without Clock	3290	1000.115	10.561
2	With Clock	3307	62.447	7.406

5. Conclusion

Fast Fourier Transform (FFT) is considered to be at the core of digital signal processing and communication systems. However, it is also considered to have high resource requirements. In this article, the authors have proposed a novel low-power and low-complexity architecture of the FFT algorithm that makes use of an innovative input grouping method to minimize the number of multiplications between the inputs and the twiddle factors of the FFT. In addition to this, a partial sum sharing method is also presented to minimize hardware resource

requirements. Logic synthesis results of the ASIC design of the proposed 32-point FFT architecture have been presented to validate the efficiency of the proposed method.

References

- [1] W. Hong et al., “Multibeam antenna technologies for 5G wireless communications,” *IEEE Trans. Antennas Propag.*, vol. 65, no. 12, pp. 6231–6249, Dec. 2017.
- [2] P. K. Daram and C.-I. H. Chen, “Smart FFT measurement for reconfigurable sensor using a wideband digital receiver,” in *Proc. IEEE Nat. Aerosp. Electron. Conf.*, Aug. 2021, pp. 200–203.
- [3] J. Chen, J. Hu, S. Lee, and G. E. Sobelman, “Hardware efficient mixed radix-25/16/9 FFT for LTE systems,” *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 23, no. 2, pp. 221–229, Feb. 2015.
- [4] N. Lei and X. Gu, “FFT-OT: A fast algorithm for optimal transportation,” in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2021, pp. 6260–6269.
- [5] J. W. Cooley and J. W. Tukey, “An algorithm for the machine calculation of complex Fourier series,” *Math. Comput.*, vol. 19, pp. 297–301, Apr. 1965.
- [6] J. Wang, S. Li, and X. Li, “Scheduling of data access for the radix-2k FFT processor using single-port memory,” *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 28, no. 7, pp. 1676–1689, Jul. 2020.
- [7] J. Hazarika, M. T. Khan, and S. R. Ahamed, “Low-complexity continuous-flow memory-based FFT architectures for real-valued signals,” in *Proc. 32nd Int. Conf. VLSI Des. and 18th Int. Conf. Embedded Syst. (VLSID)*, Jan. 2019, pp. 46–51.
- [8] S. Liu and D. Liu, “A high-flexible low-latency memory-based FFT processor for 4G, WLAN, and future 5G,” *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 27, no. 3, pp. 511–523, Mar. 2019.
- [9] T. Kawabata and H. Tsutsui, “An evaluation of high-throughput scalable radix-4 FFT processor architecture using fixed-point arithmetic,” in *Proc. Asia-Pacific Signal Inf. Process. Assoc. Annu. Summit Conf. (APSIPA ASC)*, Dec. 2020, pp. 114–117.
- [10] N. Nadar, M. Mehta, K. Bhat, J. Mendes, and R. Chaudhari, “Hardware implementation of pipelined FFT using polyphase decomposition,” in *Proc. Int. Conf. Recent Trends Electron. Inf. Commun. Technol.*, Aug. 2021, pp. 339–343.
- [11] M. Parfieniuk, “Lifting-based alternatives to the FFT for computing the 4-, 8-, and 16-point discrete Fourier transforms,” in *Proc. 29th Eur. Signal Process. Conf. (EUSIPCO)*, Aug. 2021, pp. 2184–2188.
- [12] IEEE Standard for Information Technology—Telecommunications and Information Exchange Between Systems—Local and Metropolitan Area Networks—Specific Requirements—Part 11: Wireless LAN Medium Access Control (MAC) and Physical

Layer (PHY) Specifications Amendment 3: Enhancements for Very High Throughput in the 60 GHz Band, IEEE Standard 802.11ad-2012, Dec. 2012.

- [13]B. Liu et al., “Precision adaptive MFCC based on R2SDF-FFT and approximate computing for low-power speech keywords recognition,” *IEEE Circuits Syst. Mag.*, vol. 21, no. 4, pp. 24–39, 4th Quart., 2021.
- [14]H.-Y. Chin, P.-Y. Tsai, and S.-Y. Lee, “Implementation of a two-dimensional FFT/IFFT processor for real-time high-resolution synthetic aperture radar imaging,” in *Proc. IEEE Workshop Signal Process. Syst. (SiPS)*, Oct. 2021, pp. 211–216.
- [15]S.-C. Hsu, S.-J. Huang, S.-G. Chen, S.-C. Lin, and M. Garrido, “A 128-point multi-path SC FFT architecture,” in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, Oct. 2020, pp. 1–5.
- [16]M. Garrido and P. Paz, “Optimum MDC FFT hardware architectures in terms of delays and multiplexers,” *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 68, no. 3, pp. 1003–1007, Mar. 2021.
- [17]X. Han, J. Chen, and S. Rahardja, “A new twiddle factor merging method for low complexity and high speed FFT architecture,” in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, Sep. 2019, pp. 1–5.
- [18]X. Han, J. Chen, B. Qin, and S. Rahardja, “A novel area-power efficient design for approximated small-point FFT architecture,” *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 39, no. 12, pp. 4816–4827, Dec. 2020.
- [19]T.-W. Siu, C.-W. Sham, and F. C. M. Lau, “Operating frequency improvement on FPGA implementation of a pipeline large-FFT processor,” in *Proc. 19th Int. Conf. Adv. Commun. Technol. (ICACT)*, Feb. 2017, pp. 5–9.
- [20]T.-T. Hoang, D.-H. Le, and C.-K. Pham, “VLSI design of floating-point twiddle factor using adaptive CORDIC on various iteration limitations,” in *Proc. IEEE 12th Int. Symp. Embedded Multicore/Many-Core Syst. Chip*, Sep. 2018, pp. 225–232.
- [21]W. M. Gentleman and G. Sande, “Fast Fourier transforms: For fun and profit,” in *Proc. AFIPS Fall Joint Comput. Conf.*, San Francisco, CA, USA, Nov. 1966, pp. 563–578.
- [22]J. Ding, J. Chen, and C.-H. Chang, “A new paradigm of common subexpression elimination by unification of addition and subtraction,” *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 35, no. 10, pp. 1605–1617, Oct. 2016.
- [23]M. Garrido and P. Malagón, “The constant multiplier FFT,” *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 68, no. 1, pp. 322–335, Jan. 2021.

